

C programming basics

Yuning You

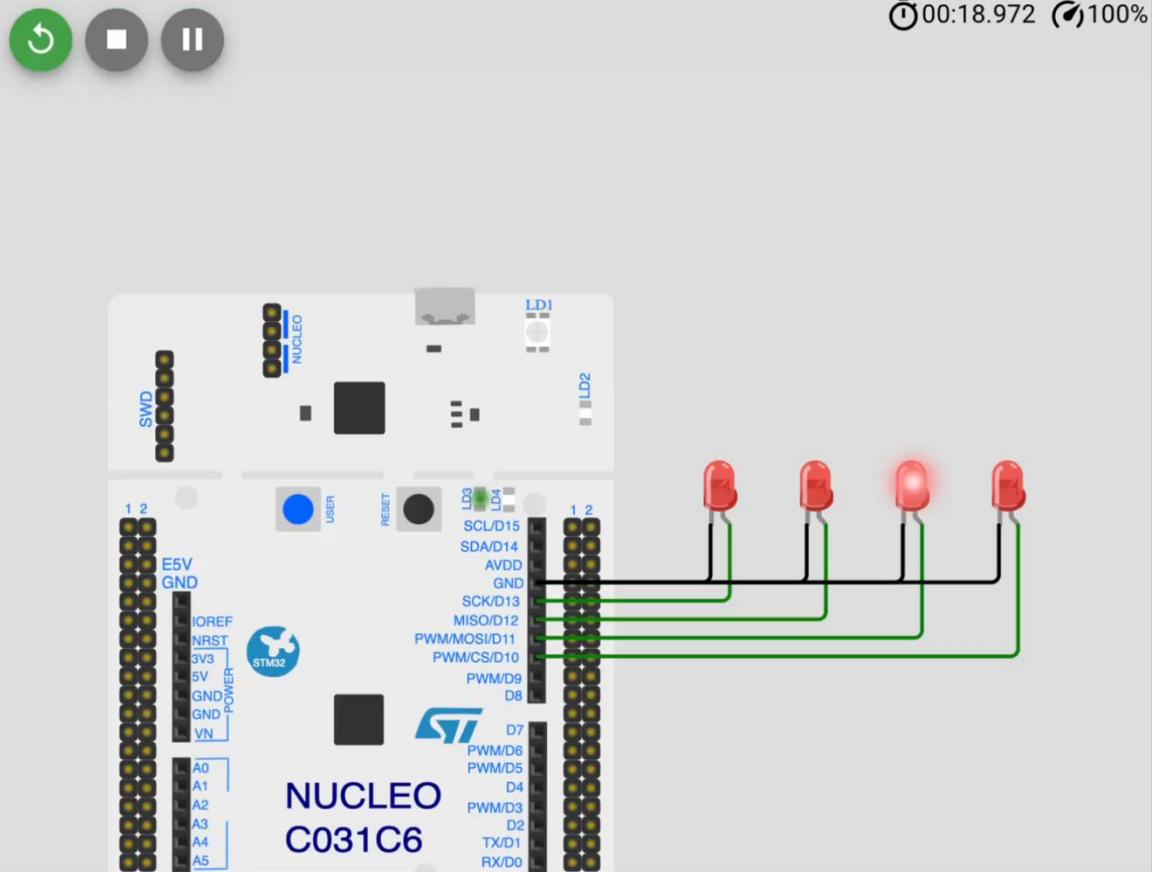
School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: Running LEDs

- A running LED demo:

<https://wokwi.com/projects/447390165530456065>

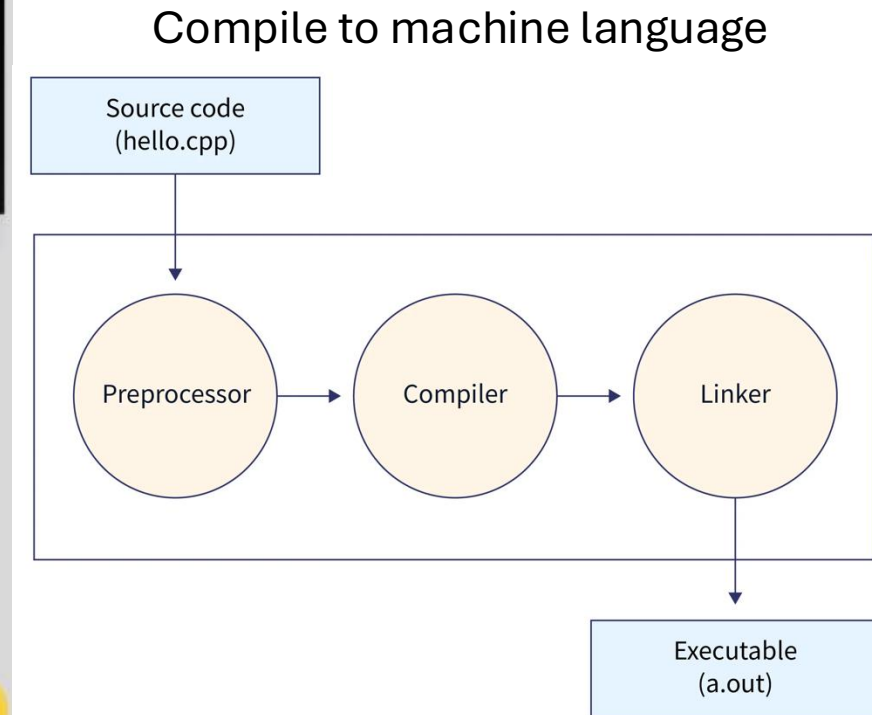
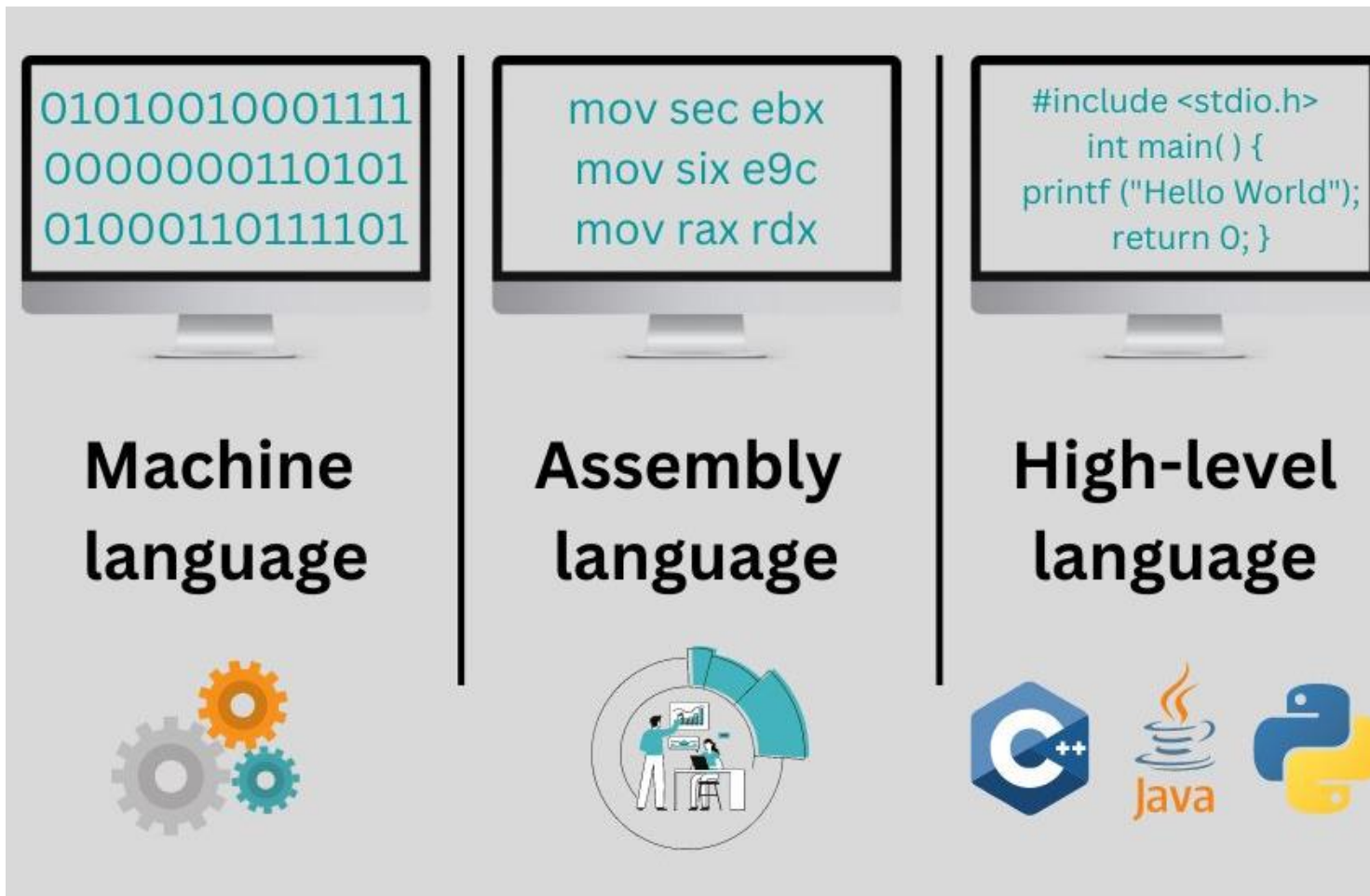
```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];          // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};   // store current state (LOW/HIGH)
7 unsigned long currTime;                   // current time variable
8
9 void setup() {
10     currTime = 0;                         // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);        // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();      // record initial time
15     }
16 }
17
18 void loop() {
19     currTime = millis();                  // get current time
20
21     for (int i = 0; i < 4; i++) {
22         // check all LEDs
23         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
24             // time to toggle the LED
25             ledState[i] = !ledState[i];    // flip LED state
26             digitalWrite(ledPin[i], ledState[i]); // apply new state
27             lastChangeTime[i] = currTime;   // reset timer for that LED
28         }
29     }
30 }
```



The image shows a Wokwi simulation of a NUCLEO C031C6 board. The board is connected to four red LEDs via jumper wires. The LEDs are connected to pins 13, 12, 11, and 10. The simulation interface includes a timer at 00:18.972 and a 100% zoom level. The code on the left is a 4-bit binary counter using LEDs, where each LED toggles at a different rate (8000ms, 4000ms, 2000ms, and 1000ms).

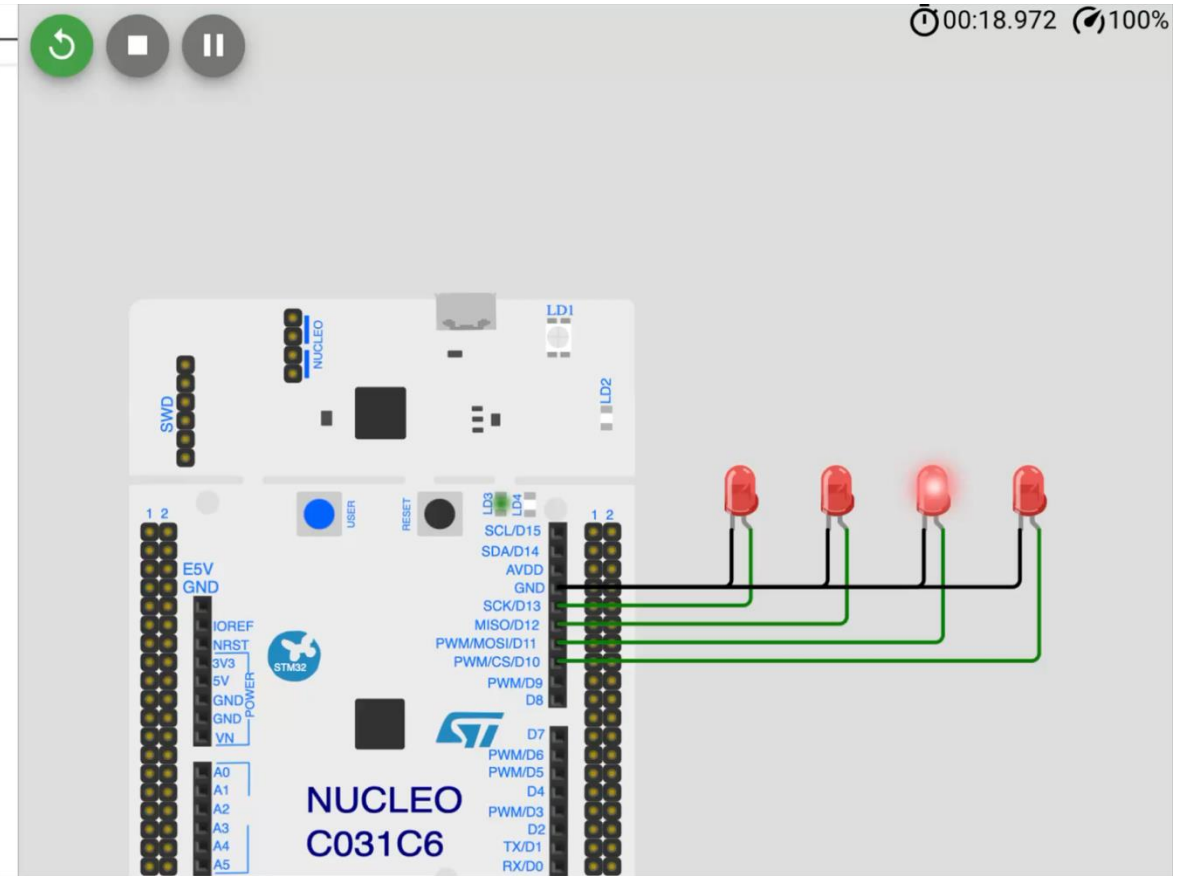
Assembly vs C programming language

- The bridge between human and machines



Fundamental control logics in programming

```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];           // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};    // store current state (LOW/HIGH)
7 unsigned long currTime;                     // current time variable
8
9 void setup() {
10     currTime = 0;                           // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);          // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();        // record initial time
15     }
16 }
17
18 void loop() {
19     currTime = millis();                     // get current time
20
21     for (int i = 0; i < 4; i++) {           // check all LEDs
22         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
23             // time to toggle the LED
24             ledState[i] = !ledState[i];      // flip LED state
25             digitalWrite(ledPin[i], ledState[i]); // apply new state
26             lastChangeTime[i] = currTime;    // reset timer for that LED
27         }
28     }
29 }
30
```



Basic control structure: While loop

```
int i = 0;  
do {  
    printf("%d ", i);  
} while(i++ < 3);
```

=> 0 1 2 3

Basic control structure: For loop

```
// C99-style  
for(int i = 0; i < 3; ++i){  
    printf("%d ", i);  
}
```

=> 0 1 2

Basic control structure: If else condition

```
int i = 0;
if(i < 3){
    printf("It sure is.");
} else if(i == 3){
    printf("Nope.");
} else {
    printf("Still nope.");
}
```

Basic control structure: Switch condition

```
● switch(i){  
    case 1:  
        printf("It's one!");  
        break;  
    case 2:  
        printf("It's two!!");  
        break;  
    default:  
        printf("It's something else!!!");  
}
```


Basic control structures: Jumps

```
void foo(){
    for(int i = 0; i < 10; ++i){
        printf("%d ", i);
        if(i == 2){
            i = 3;
            continue;
        } else if(i == 6){
            break;
        }
    }

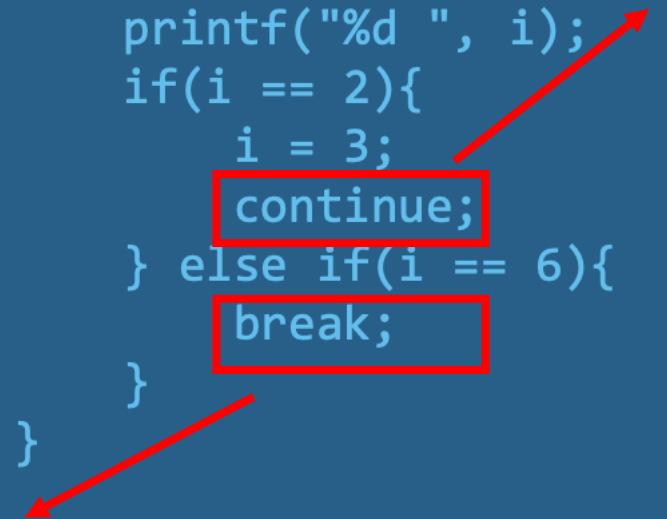
    goto end;
    printf("near the end\n");
end:
    printf("the end\n");
    return;
    printf("or is it?");
}
```

Output:

0 1 2 4 5 6 the end

Basic control structures: Jumps

```
void foo(){
    for(int i = 0; i < 10; ++i){
        printf("%d ", i);
        if(i == 2){
            i = 3;
            continue;
        } else if(i == 6){
            break;
        }
    }
    goto end;
    printf("near the end\n");
end:
    printf("the end\n");
    return;
    printf("or is it?");
}
```



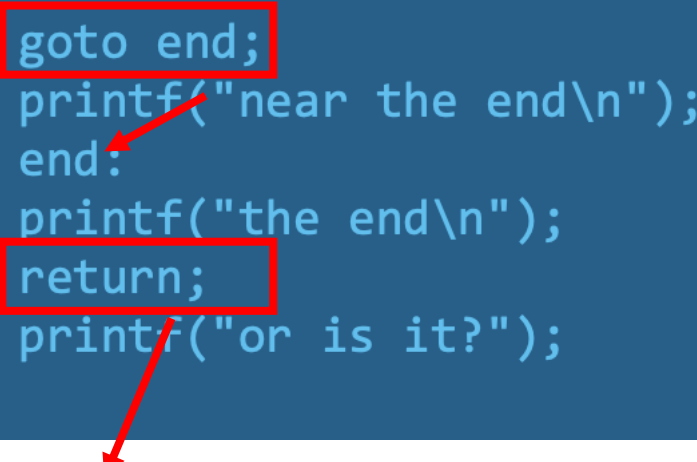
Output:

0 1 2 4 5 6 the end

Basic control structures: Jumps

```
void foo(){
    for(int i = 0; i < 10; ++i){
        printf("%d ", i);
        if(i == 2){
            i = 3;
            continue;
        } else if(i == 6){
            break;
        }
    }

    goto end;
    printf("near the end\n");
end:
    printf("the end\n");
    return;
    printf("or is it?");
}
```



Output:

0 1 2 4 5 6 the end

About the goto statement

- Syntax: `goto label`
 - ... where *label* refers to an earlier or later labelled section of code.
- Target label must be in the same function as the goto statement.
- Notorious for creating hard-to-read code, but the concept is critical to how computers operate.

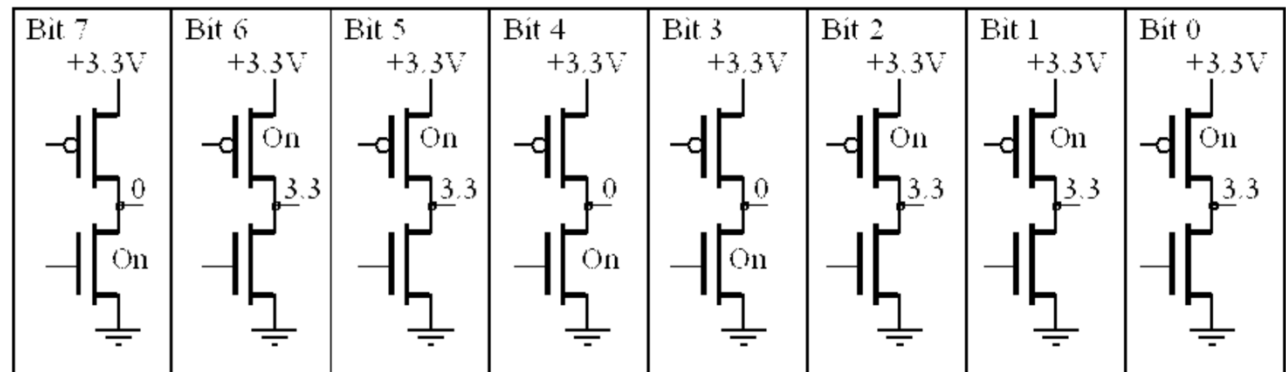
Variable types

```
int i = 0;
do {
    printf("%d ", i);
} while(i++ < 3);
```

=> 0 1 2 3

Data Type	Size (in bytes)	Range	Format Specifiers
char	1	-128 to 127	%c
int	2	-32768 to 32767	%d
float	4	3.4×10^{-38} to 3.4×10^{38}	%f
double	8	1.7×10^{-308} to 1.7×10^{308}	%lf

A byte implemented with transistors



Variable and constant

```
int a = 1;  
a = 2; // cool
```

```
const int b = 1;  
b = 2;  
// error:  
// read-only variable  
// is not assignable
```

Static variable

- Static variables retain their value throughout the life of the program

```
void foo(){  
    static int count = 0;  
    printf("%d ", count++);  
}  
  
for(int i = 0; i < 5; ++i){  
    foo();  
}
```

Output: 0 1 2 3 4

Arithmetic operator

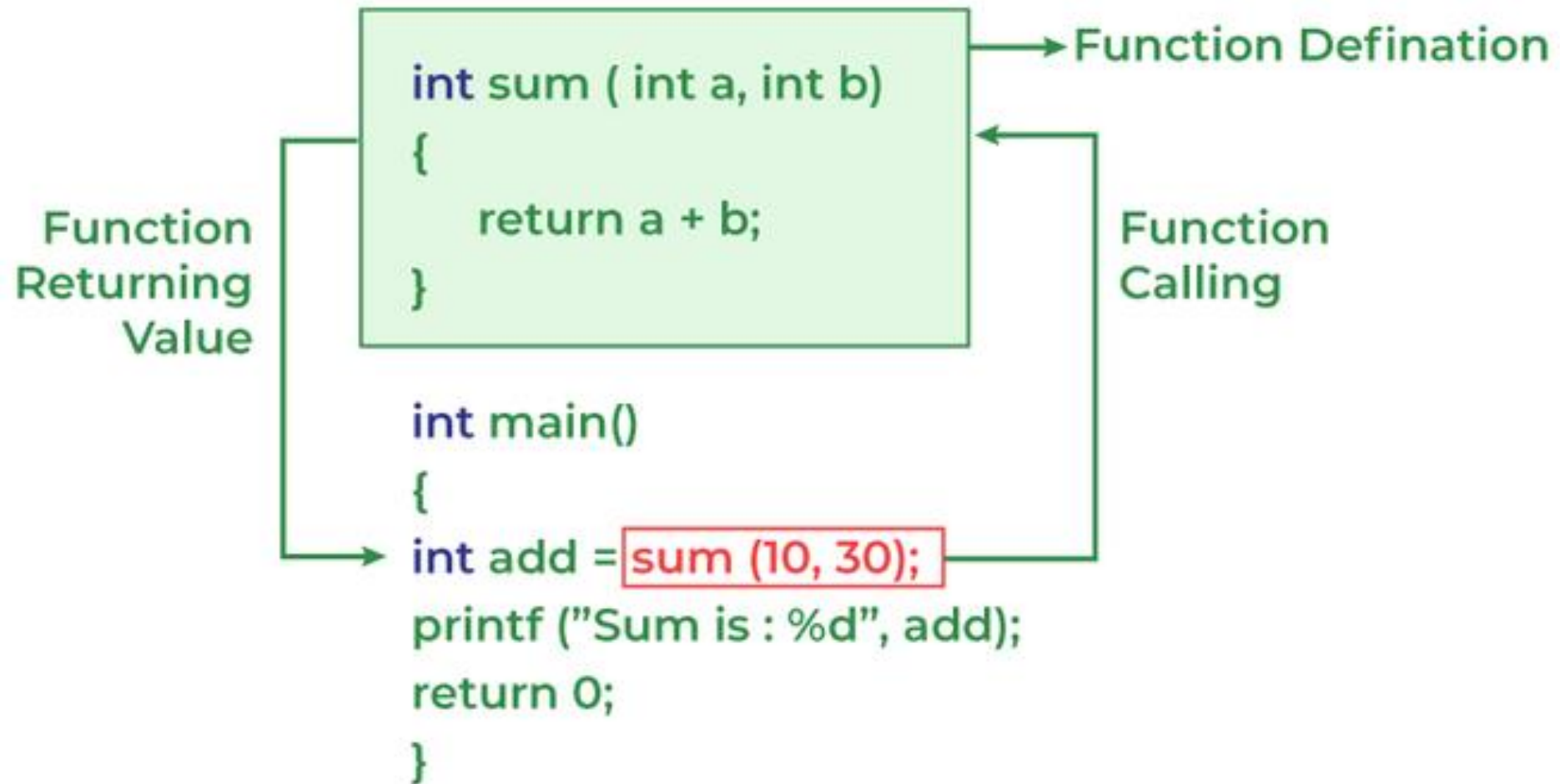
<i>Operation</i>	<i>Meaning</i>
=	Assignment statement
?	Selection
<	Less than
>	Greater than
!	Logical not (true to false, false to true)
~	1's complement
+	Addition
-	Subtraction
*	Multiply or pointer reference
/	Divide
%	Modulo, division remainder
	Logical or
&	Logical and, or address of
^	Logical exclusive or
.	Used to access parts of a structure

<i>Operation</i>	<i>Meaning</i>
==	Equal to comparison
<=	Less than or equal to
>=	Greater than or equal to
!=	Not equal to
<<	Shift left
>>	Shift right
++	Increment
--	Decrement
&&	Boolean and
	Boolean or
+=	Add value to
-=	Subtract value to
*=	Multiply value to
/=	Divide value to
=	Or value to
&=	And value to
^=	Exclusive or value to
<<=	Shift value left
>>=	Shift value right
%=	Modulo divide value to
->	Pointer to a structure

Arithmetic operator

```
void main(void) {  
    long x,y,z;    // Three local variables  
    x=1; y=2;    // set the values of x and y  
    z = x+4*y;    // arithmetic operation  
    x++;          // same as x=x+1;  
    y--;          // same as y=y-1;  
    x = y<<2;     // left shift same as x=4*y;  
    z = y>>2;     // right shift same as x=y/4;  
    y += 2;       // same as y=y+2;  
}
```

Function



Function

```
// Calculates area
// Input:  side of a room (unsigned long) in meters
// Output: area of the room (unsigned long) in square meters
unsigned long Calc_Area(unsigned long s) {
    unsigned long result;
    result = s*s;
    return(result);
}
int main(void) {
    unsigned long side; // room wall meters
    unsigned long area; // size squared meters
    UART_Init(); // call subroutine to initialize the uart
    printf("This program calculates areas of square-shaped rooms\n");
    side = 3;
    area = Calc_Area(side);
    printf("\nArea of the room with side of %ld m is %ld sqr m\n",side,area);
    side = side+2;
    area = Calc_Area(side);
    printf("\nArea of the room with side of %ld m is %ld sqr m\n",side,area);
}
```

Scope

```
void bar(){
    int a = 0;
    if(3 > 0){
        int b = 0;
        b = 2; // okay
    }
    a++; // okay
    b++; // error:
        // use of undeclared
        // identifier 'b'
}
```

```
void foo(){
    int a = 0;
}
```

In **foo** and **bar**,
a is "in scope" for
the entire function.
b is "in scope" only within
the if statement's block in **bar**.

Avoid these situations

- Uninitialized variables:

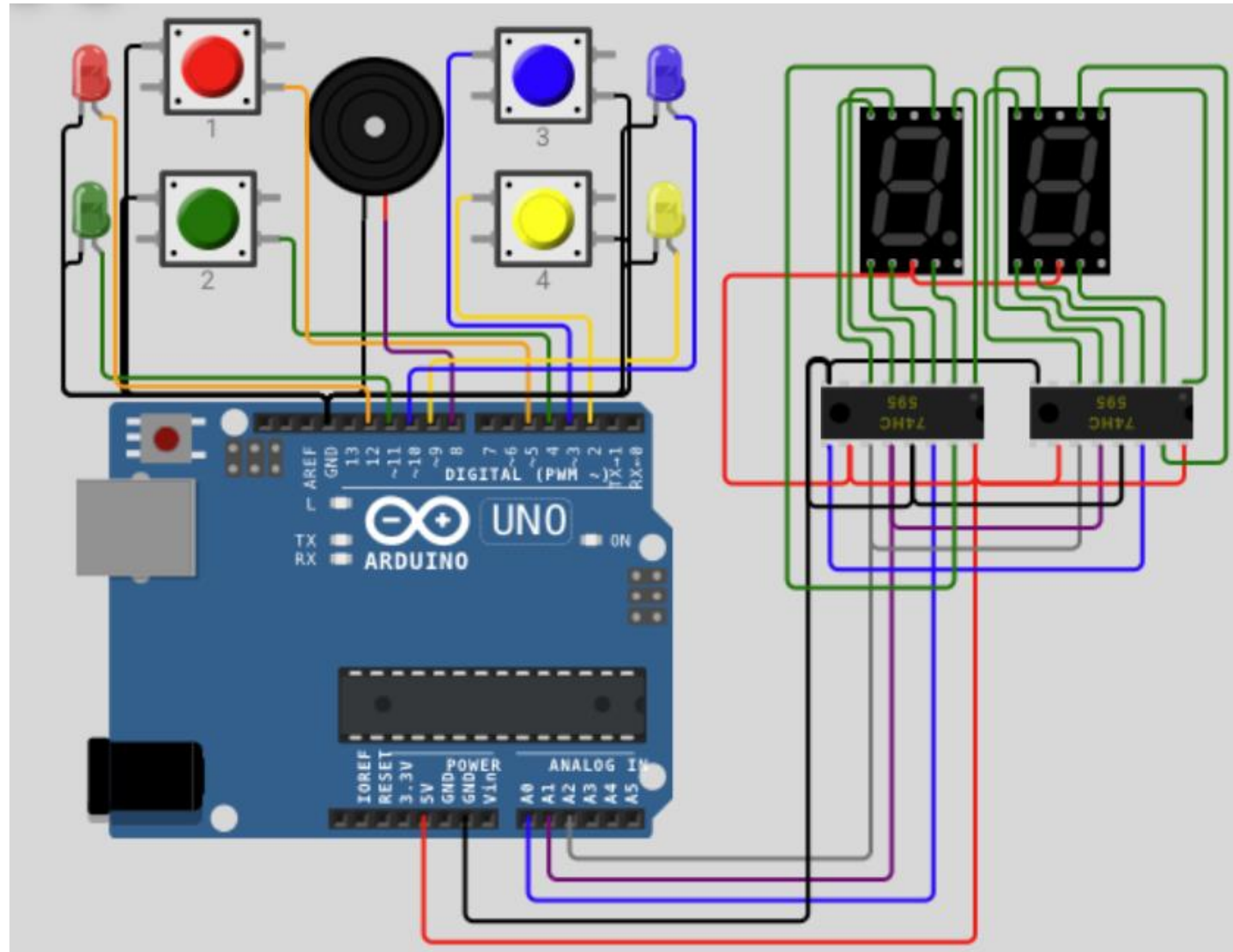
```
int i;  
printf("%d", i);
```

- Out-of-bounds array access:

```
char reversed[20];  
char out_of_bounds = reversed[21];
```

- Variables passed out of their defining function's scope.

Virtual lab session: Simon game



Virtual lab session: Simon game

- Can you write down the logic of how the Simon game works in C-style?

Virtual lab session: Simon game

- Can you add a constant time limit for each player's response?

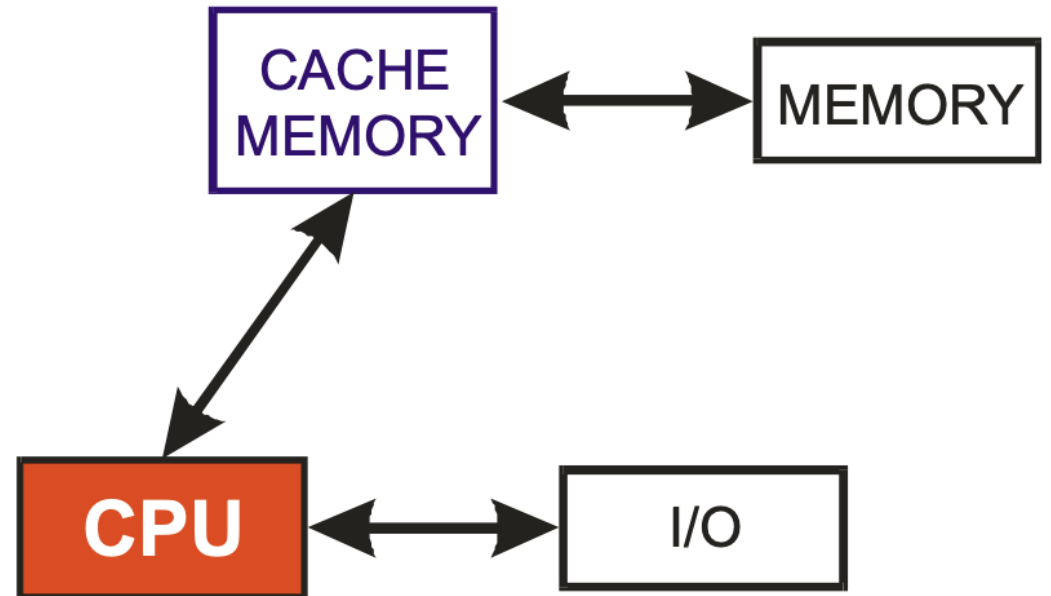
Recap: Variable and constant and everything take memory

```
int a = 1;
```

```
const int b = 1;
```

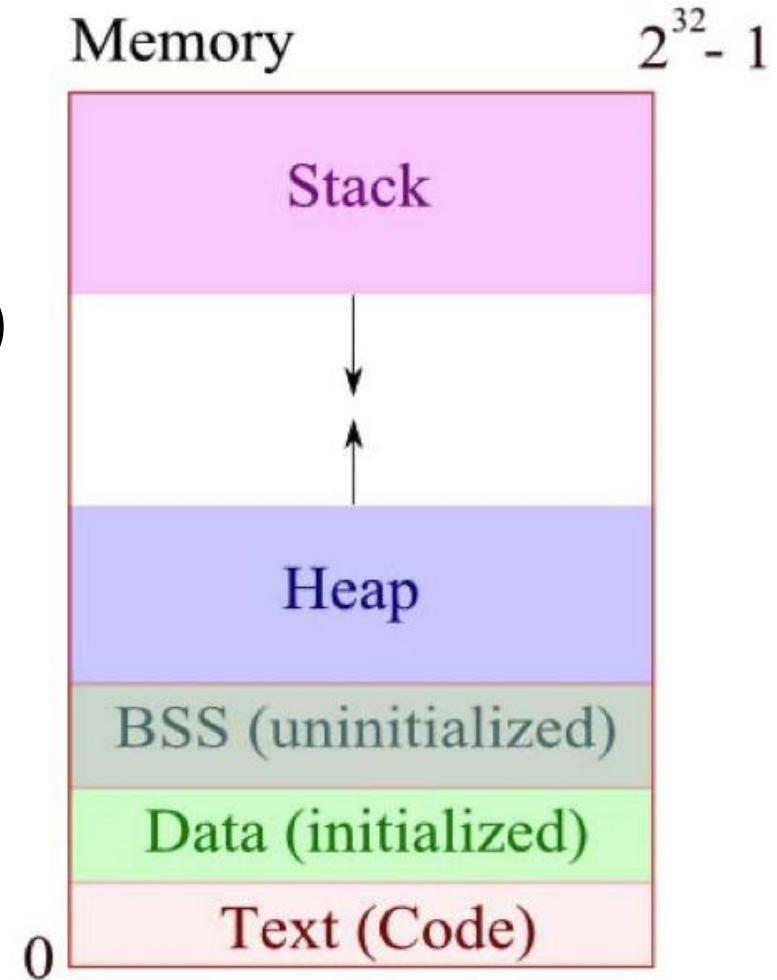
Memory:

```
00000000 00000000 00  
00000000 00000000 00  
00000000 00000000 00  
00000000 00000000 00
```



Memory layout

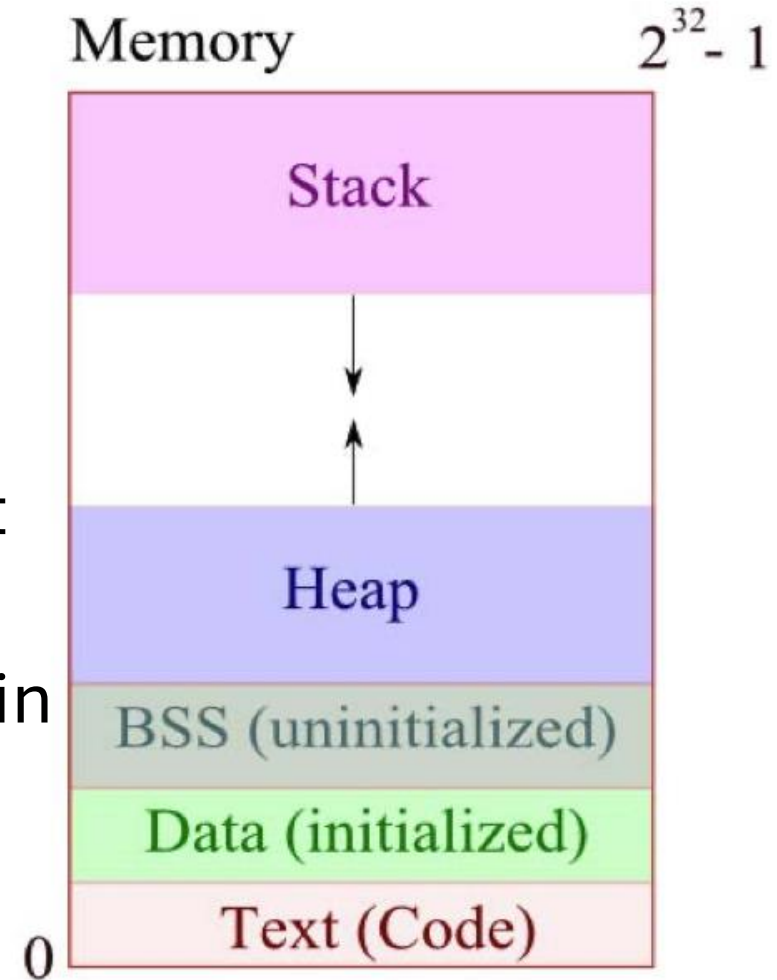
- Text Segment (.text)
- Purpose: Stores the executable instructions (i.e., your compiled code)
- Read-only: Prevents accidental modification of program code
- Shared: Can be shared between multiple processes running the same program
- Also called: Code segment.



Memory Layout diagram courtesy of bogotobogo.com, and used with permission.

Memory layout

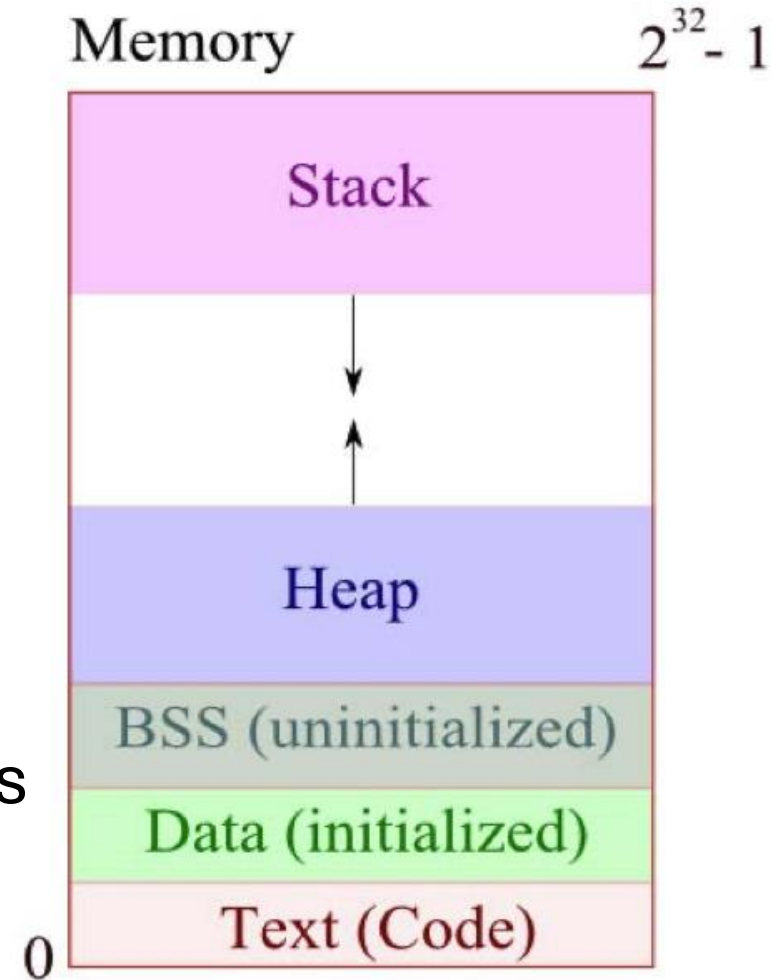
- Initialized Data Segment (.data)
- Purpose: Holds global and static variables that are explicitly initialized with a non-zero (or non-default) value
- Read-write: Values can be modified at runtime
- Stored in executable: Takes up space in the binary file because initial values must be saved



Memory Layout diagram courtesy of bogotobogo.com, and used with permission.

Memory layout

- Uninitialized Data Segment (.bss — Block Started by Symbol)
- Purpose: Holds global and static variables that are uninitialized or initialized to zero
- Read-write: Like .data, but does NOT take space in the executable file
- At program load time, the OS allocates memory for .bss and zeros it out



Memory Layout diagram courtesy of bogotobogo.com, and used with permission.

Heap

- Heap is a chunk of memory that users can use to dynamically allocated memory
- Memory is manually managed by the programmer (e.g., using malloc/free in C or new/delete in C++)
- Lasts until freed, or program exits

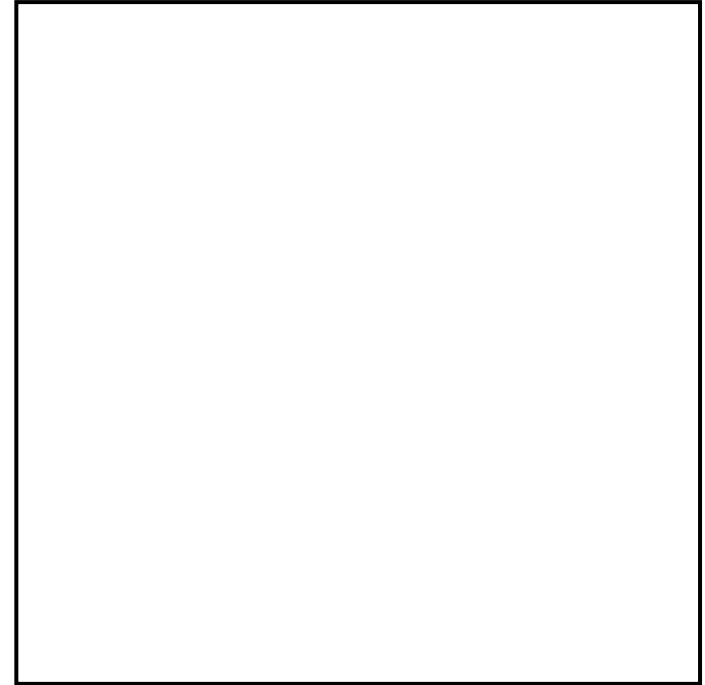
Stack

- Memory is automatically allocated and deallocated by the compiler
- Used for storing local variables, function parameters, and return addresses
- Allocation is done in a LIFO (Last In, First Out) manner.

Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

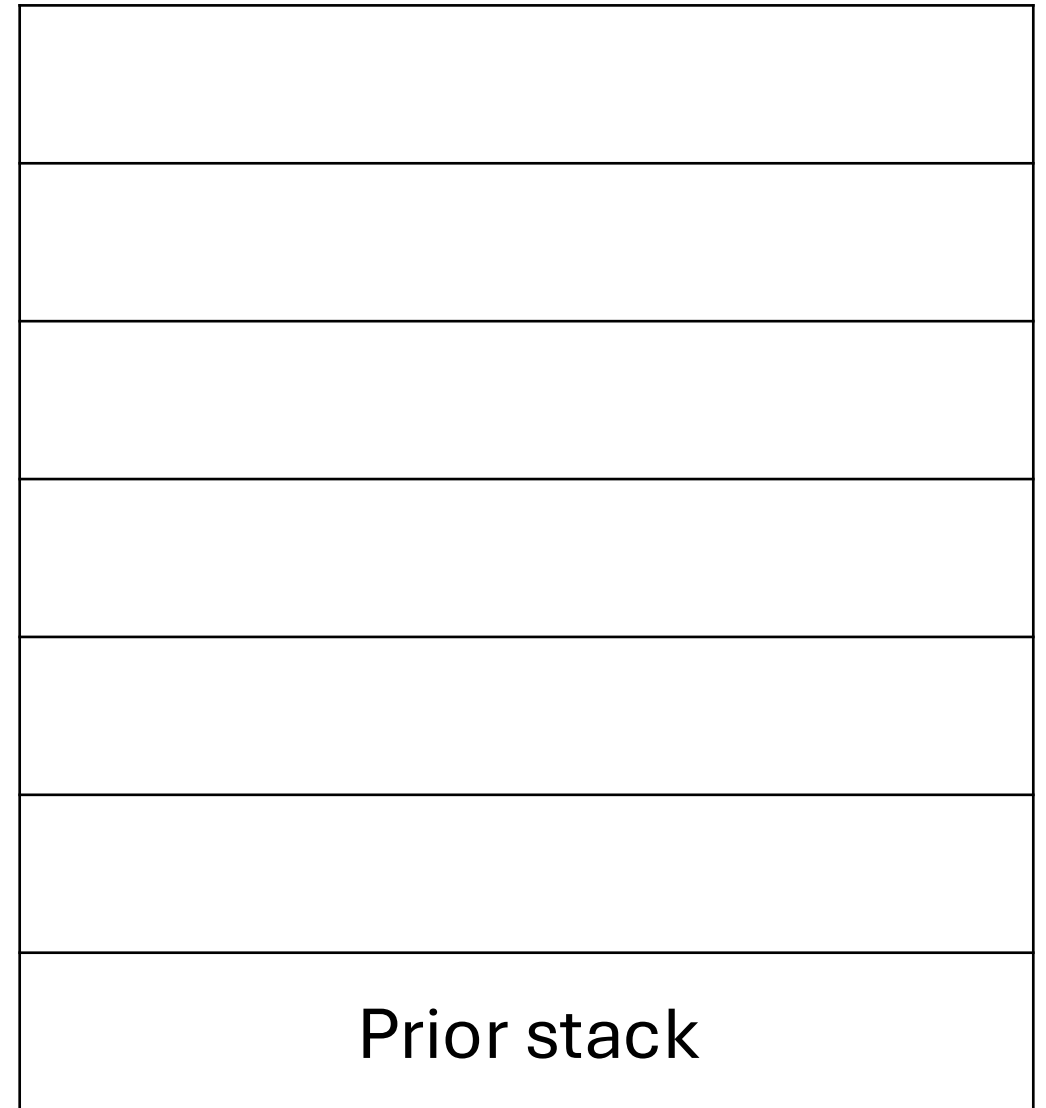
```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```



Call stack

```
void DrawSquare(int i){
    int start, end, ...
    DrawLine(start, end);
}
```

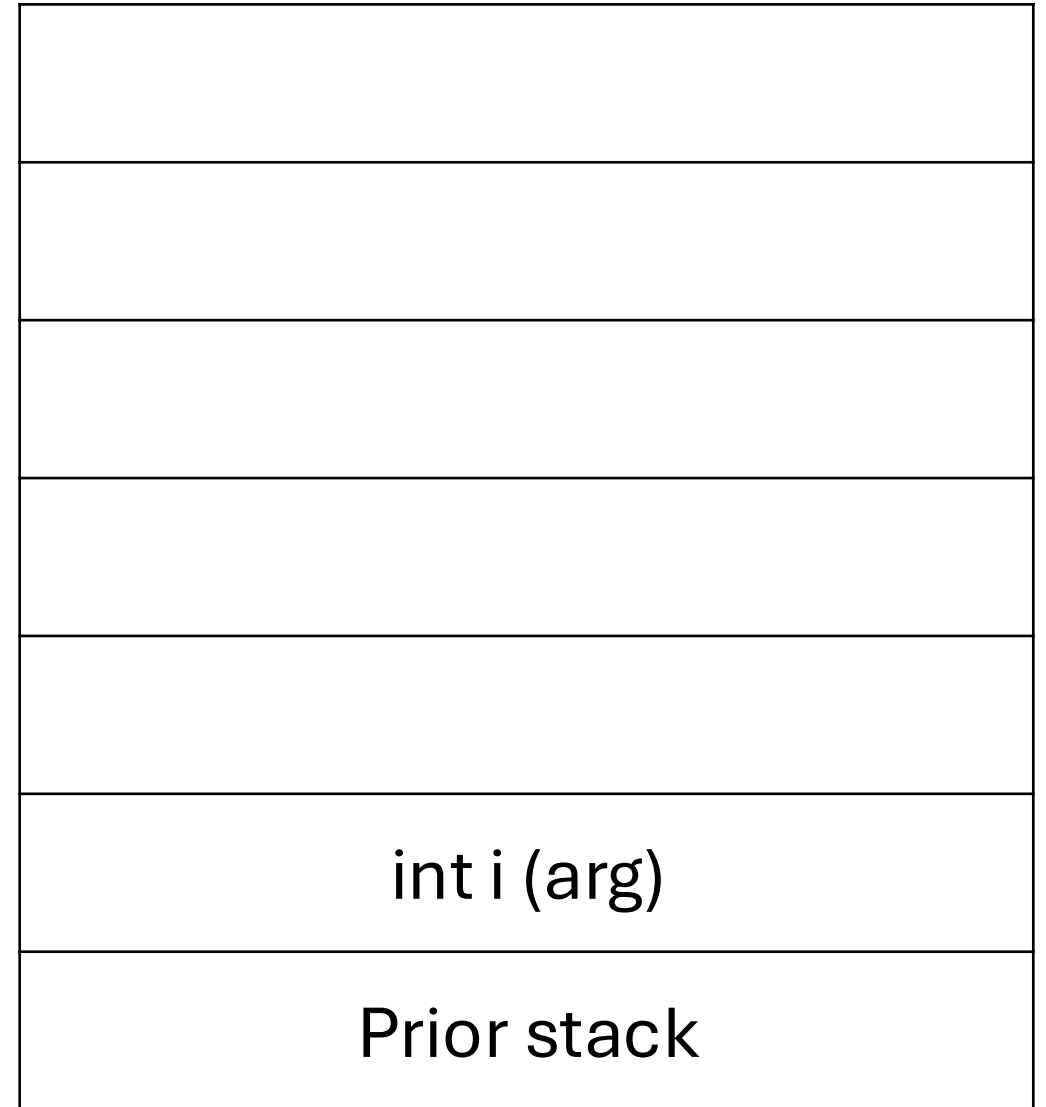
```
void DrawLine(int start, int end){
    //local variables
    ...
}
```



Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

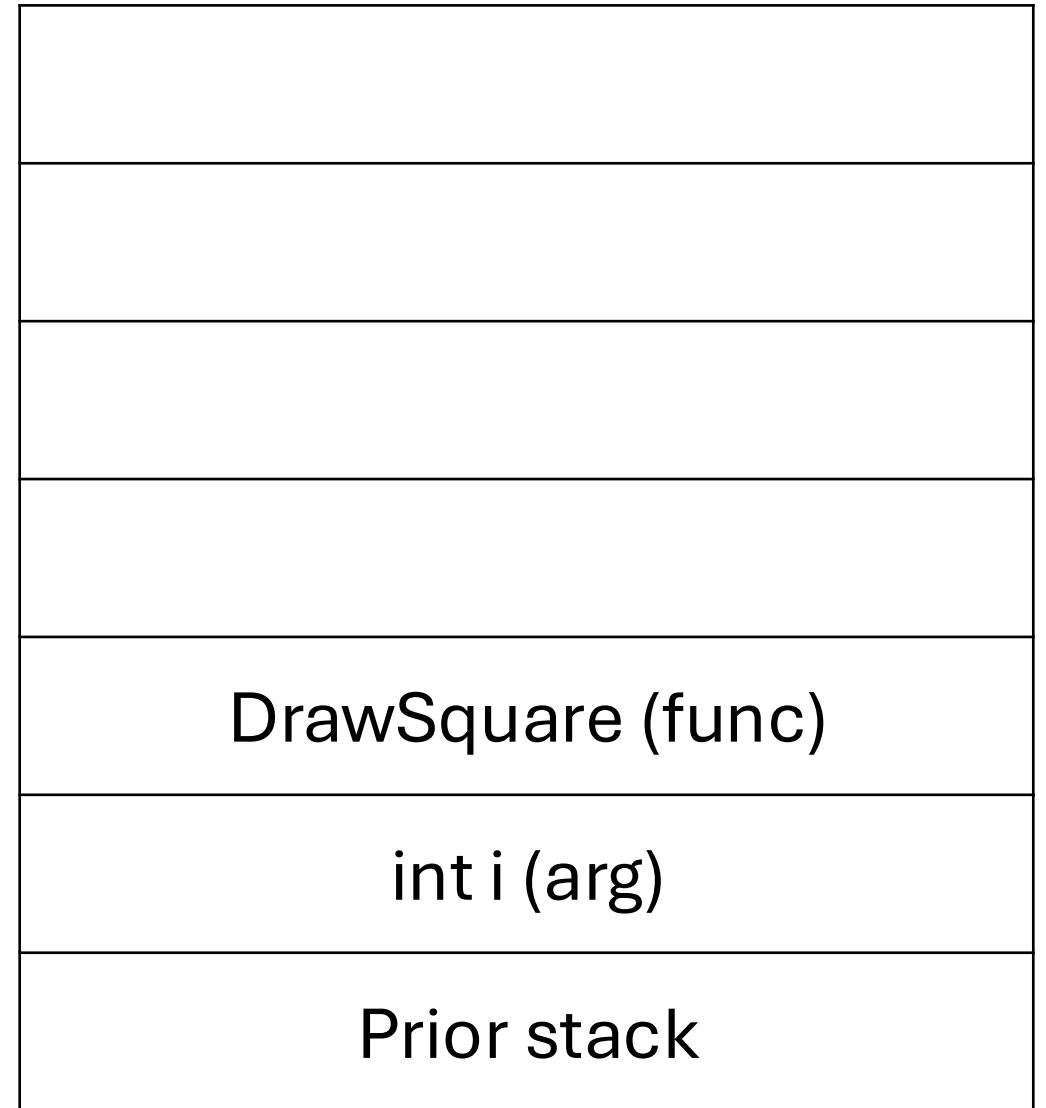
```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```



Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

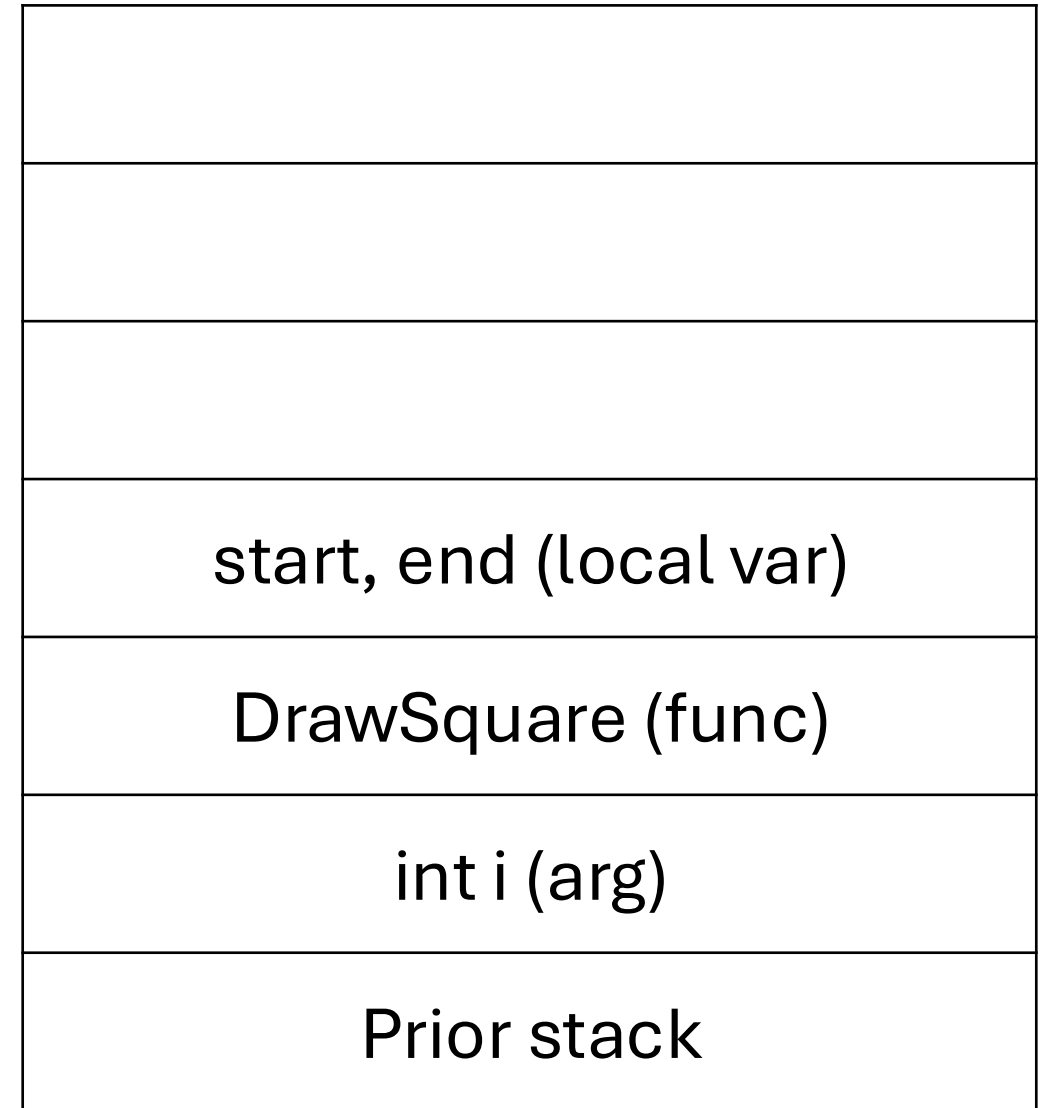
```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```



Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```



Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```

start, end (arg)
start, end (local var)
DrawSquare (func)
int i (arg)
Prior stack

Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```

DrawLine (func)
start, end (arg)
start, end (local var)
DrawSquare (func)
int i (arg)
Prior stack

Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```

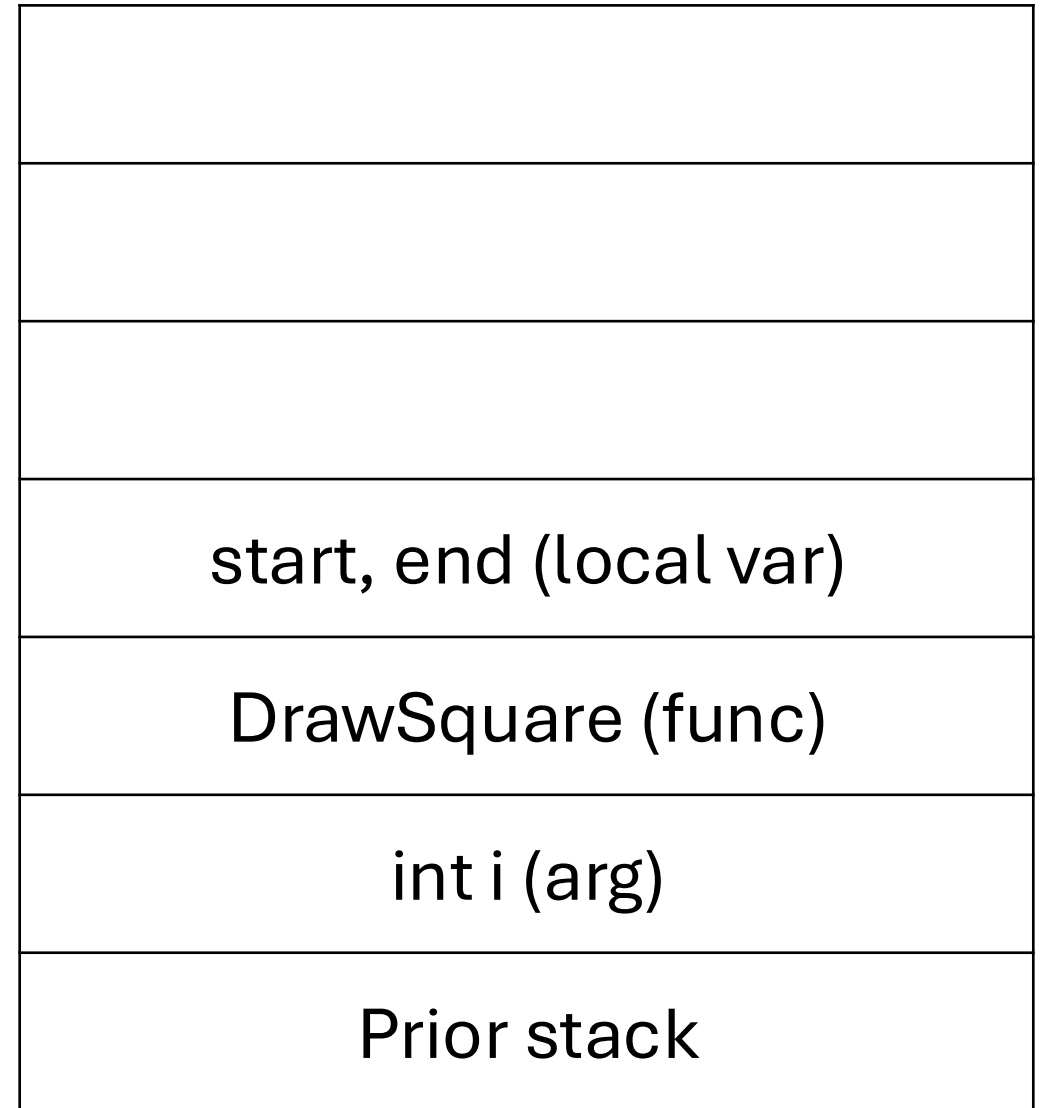
DrawLine local var
DrawLine (func)
start, end (arg)
start, end (local var)
DrawSquare (func)
int i (arg)
Prior stack

Call stack

```
void DrawSquare(int i){  
    int start, end, ...  
    DrawLine(start, end);  
}
```

return

```
void DrawLine(int start, int end){  
    //local variables  
    ...  
}
```

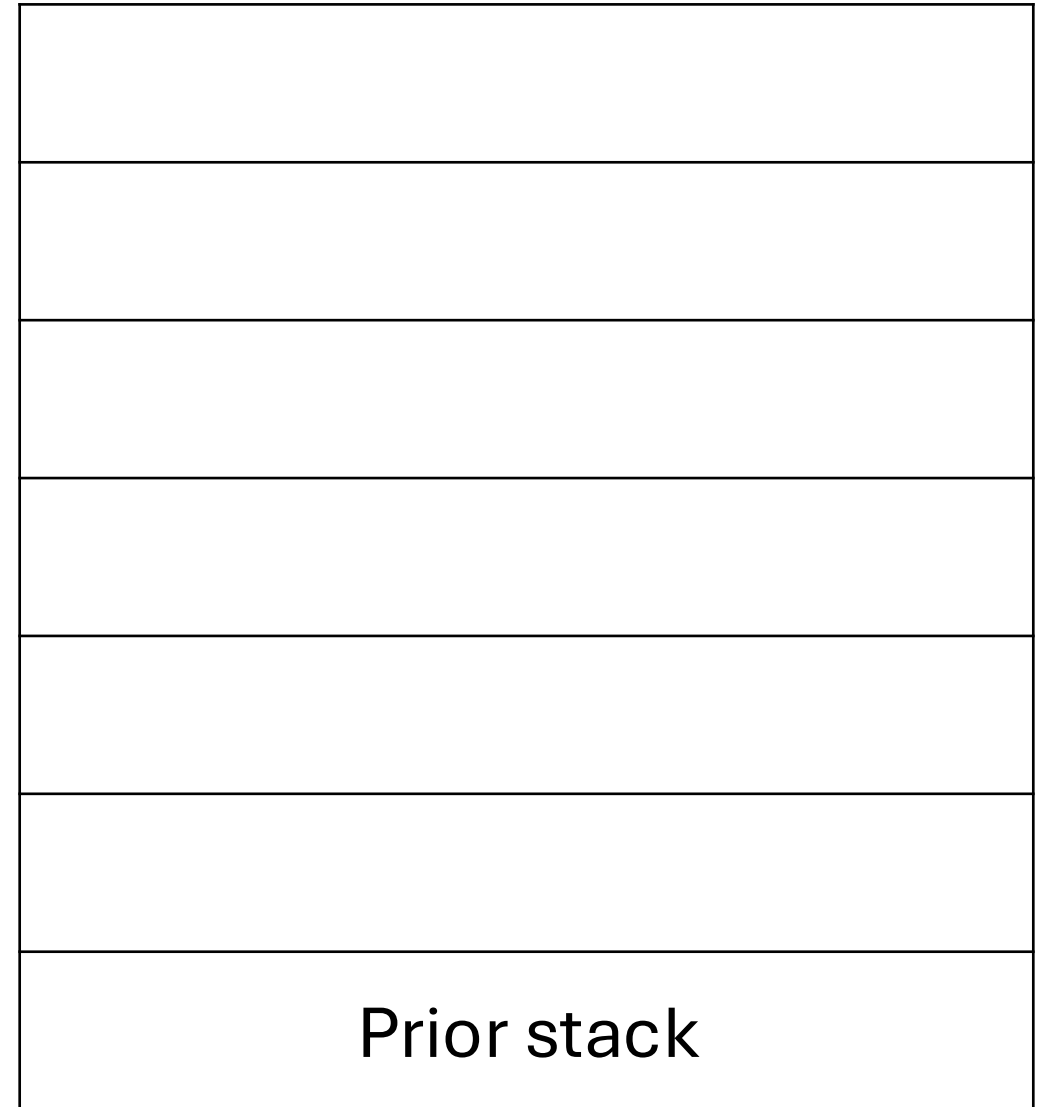


Call stack

return

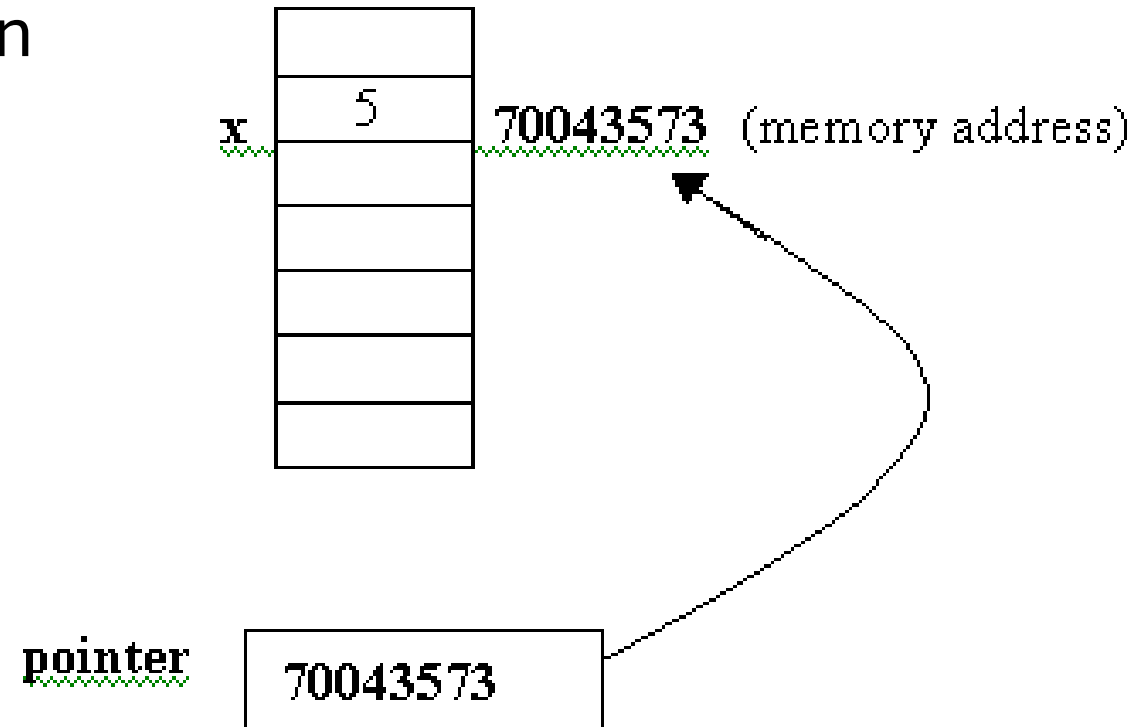
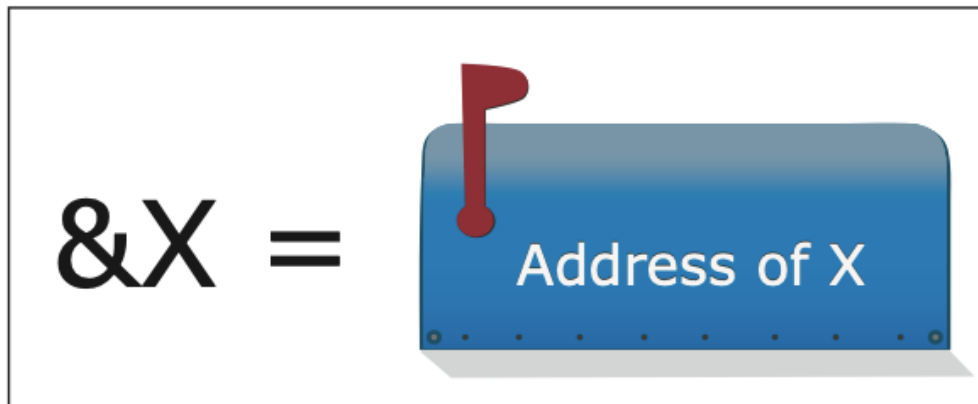
```
void DrawSquare(int i){
    int start, end, ...
    DrawLine(start, end);
}
```

```
void DrawLine(int start, int end){
    //local variables
    ...
}
```



Pointers and addresses

- Address: Each variable represents an address in memory and a value
- Address:
 - `&variable` = address of variable



Pointers and addresses

- A pointer is a variable that “points” to the block of memory that a variable represent.
 - Declaration: `data_type *pointer_name;`
 - Example:
`char x = 'a';`
`char *ptr = &x; // ptr points to a char x`
 - Pointers are integer variables themselves, so can have pointer to pointers: `char **ptr;`

Pointers and addresses

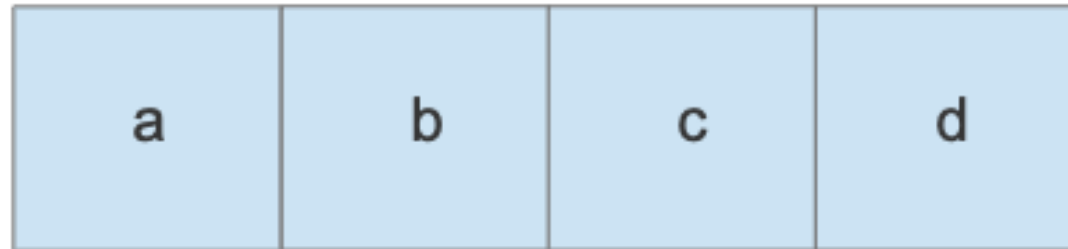
- Given pointer ptr, to get value at that address, do: *ptr
 - `int x = 5;`
`int *ptr = &x;`
`*ptr = 6; // Access x via ptr, and changes it to 6`
`printf("%d", x); // Will print 6 now`

Pointer arithmetic

- Can do math on pointers

– Ex: `char* ptr`

ptr + 1



Pointer arithmetic

- Can do math on pointers
 - $p1 = p2$: sets $p1$ to the same address as $p2$
 - Addition/subtraction:
 - $p1 + c$, $p1 - c$
 - Increment/decrement:
 - $p1++$, $p1--$

Why use pointers? They are so confusing...

- Pass-by-reference rather than value.

```
void sample_func( char* str_input);
```

- Manipulate memory effectively.
- Useful for arrays (next topic).

C arrays (Statically allocated)

- Arrays are really chunks of memory!
- Declaration:
 - `Data_type array_name[num_elements];`
- Declare array size, cannot change.

C arrays (Statically allocated)

- Can initialize like:
 - `int data[] = {0, 1, 2};` //Compiler figures out size
 - `int data[3] = {0, 1, 2};`
 - `int data[3] = {1};` // `data[0] = 1`, rest are set to 0
 - `int data[3];` //Here, values in data are still junk
 - `data[0] = 0;`
 - `data[1] = 1;`
 - `data[2] = 2;`

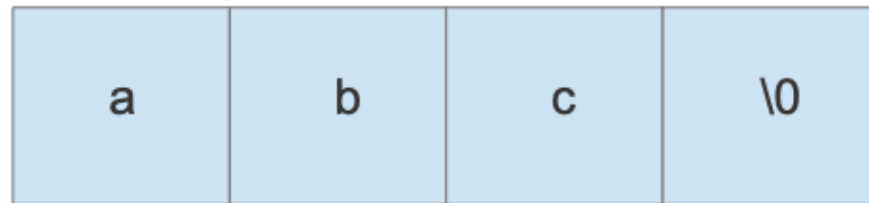
Array and pointers

- Array variables are pointers to the array start!
 - `char *ptr;`
`char str[10];`
`ptr = str; //ptr now points to array start`
`ptr = &str[0]; //Same as above line`
- Array indexing is same as dereferencing after pointer addition.
 - `str[1] = 'a'` is same as `*(str+1) = 'a'`

C-style strings

- No string data type in C. Instead, a string is interpreted as a null-terminated char array.
- Null-terminated = last char is null char `'\0'`, not explicitly written

```
char str[] = "abc";
```

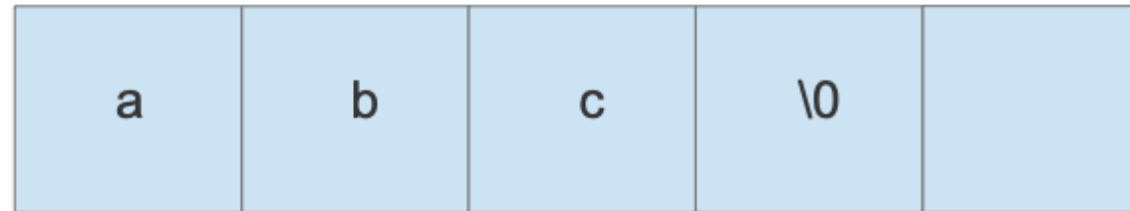


- String literals use `" "`. Compiler converts literals to char array.

C-style strings

- Char array can be larger than contained string

```
char str[5] = "abc";
```



- Special chars start with '\':
 - \n, \t, \b, \r: newline, tab, backspace, carriage return
 - \\, \', \": backslash, apostrophe, quotation mark

Dynamic allocation of memory

- `#include <stdlib.h>`
- **sizeof** (a C language keyword) returns number of bytes of a data type.
- **malloc/realloc** finds a specified amount of free memory and returns a void pointer to it.
 - `char * str = (char *) malloc( 3 * sizeof(char) );`
`strcpy(str, "hi");`
 - `str = (char *) realloc( str , 6 * sizeof(char) );`
`strcpy(str, "hello");`

Dynamic deallocation of memory

- `#include <stdlib.h>`
- **free** declares the memory pointed to by a pointer variable as free for future use:

```
char * str = (char *) malloc( 3 * sizeof(char) );  
strcpy(str, "hi");  
... use str ...  
free(str);
```

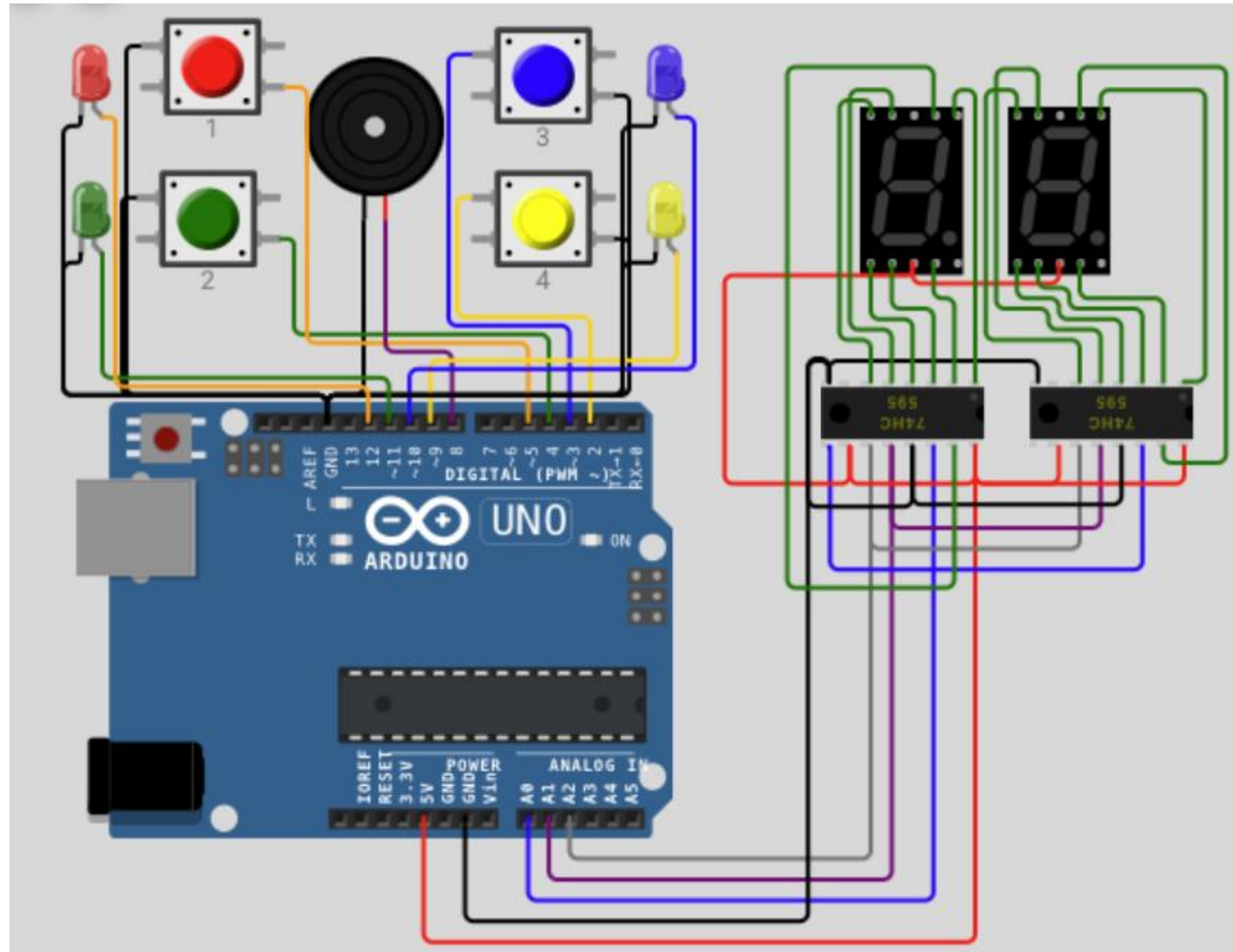
Dynamically allocated arrays

- Allows you to avoid declaring array size at declaration.
- Use malloc to allocate memory for array when needed:
 - `int *dynamic_array;`
`dynamic_array = malloc( sizeof( int ) * 10 );`
`dynamic_array[0]=1; // now points to an array`

In summary

- Memory has stack and heap.
- Pointers and addresses access memory.
- Arrays are really chunks of memory. Strings are null-terminated char arrays.
- C allows user memory allocation. Use malloc, realloc and free.

Virtual lab session recap: Simon game



Virtual lab session

- Can you change the rule from 'adding one flash each time' to 'adding two flashes each time'?

Virtual lab session

- And add a decreased time limit for each player's response?

Thank you!